Low Coupling

Low Coupling Principle

Minimize dependencies

DEFINITION

Assign responsibilities so dependencies between classes stay few and weak.

Low Coupling is an evaluative principle, not a recipe: when several designs would work, prefer the one that creates the fewest and weakest dependencies. Coupling has many forms — depending on a concrete class, reaching through an object to its neighbours, sharing global state, knowing another's internals. The goal is to keep each class leaning on as little as possible.

WHY IT MATTERS

Coupling is what makes change expensive. A tightly bound class ripples edits into everything it touches, resists reuse outside its original context, and can't be tested without dragging its collaborators along. Weakening dependencies — via interfaces, injection, and indirection — localises change and lets you swap or mock pieces freely.

Vasyl Krupka · Ukraine

— How it works

Coupling	A measure of how strongly one element depends on others — by type, by reach, by shared state.
Goal	Keep dependencies few and weak so a change ripples through as little code as possible.
Means	Program to interfaces, inject collaborators, and add indirection where two parts should not know each other.

HOW TO APPLY

1. When choosing where a method or dependency goes, list what each option would couple together.
2. Prefer depending on an interface or abstraction over a concrete class.
3. Inject collaborators instead of constructing them inside (no hidden new).
4. Stop at "few and weak" — do not chase zero coupling with needless layers.

LITMUS TEST

If this collaborator changed, how many places break? Could you unit-test the class without standing up its dependencies? Many breaks, or "no", signals coupling that is too high.

— Smell → fix

The service hard-wires a concrete repository and news it up — bound to SQL, impossible to mock:

```
class OrderService {
private repo = new SqlOrderRepo() // hidden concrete dep
place(o: Order) { this.repo.save(o) }
}
```

↓ DEPEND ON AN ABSTRACTION

```
interface OrderRepo { save(o: Order): void }
class OrderService {
private repo!: OrderRepo // injected abstraction
place(o: Order) { this.repo.save(o) }
}
```

The service depends on a small interface it owns, injected from outside. SQL, in-memory, or a mock all satisfy it — change and tests stay local.

USE WHEN

- ✓ Evaluating a design option
- ✓ Choosing where a method should live

AVOID WHEN

- ▲ Chasing zero coupling — some is necessary; don't over-indirect

— Trade-offs & pitfalls

PROS

+ Localized change
+ Reuse and testability

CONS

− Over-decoupling adds indirection and ceremony

COMMON PITFALLS

▲ Over-decoupling: interfaces and indirection for a single implementation add ceremony with no payoff.

▲ Confusing low coupling with no coupling — collaborating objects must depend on something.

▲ Maximising it alone — pushed too far it shatters cohesion into scattered, anemic classes.

WHERE YOU SEE IT

- A service depending on an OrderRepo interface, with the SQL implementation injected at the edge.
- DI containers wiring concretes so classes never new their collaborators.
- Event / message buses letting producers and consumers stay unaware of each other.

SEE ALSO

DIP	Low Coupling is the goal; DIP (depend on abstractions) is one technique to reach it
High Cohesion	the constant counter-force — balance the two, do not maximise either
LoD	Law of Demeter is a concrete rule that limits coupling to immediate neighbours
Indirection	a middleman lowers coupling — but adds parts, so apply it sparingly

Vasyl Krupka · Ukraine

— Interview & sources

Low Coupling vs DIP?

Low Coupling is the why (few, weak dependencies); DIP is one how (depend on abstractions, invert ownership of the interface). DIP serves Low Coupling.

Is zero coupling the goal?

No — zero coupling means objects that never collaborate. Some coupling is necessary; the aim is few and weak, not none. Over-decoupling adds indirection and ceremony.

Low Coupling vs High Cohesion — conflict?

They tension each other: merging everything kills cohesion, over-splitting raises coupling. Good design balances both rather than maximising one.

How do you actually measure coupling?

Look at fan-out (how many types a class names), whether it depends on concretes or abstractions, and whether you can unit-test it without its collaborators. Hard-to-mock dependencies are the practical red flag.

REMEMBER

Depend on as little as possible, as weakly as possible.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Low Coupling

Stevens, Myers & Constantine — "Structured Design" (1974): origin of coupling and cohesion